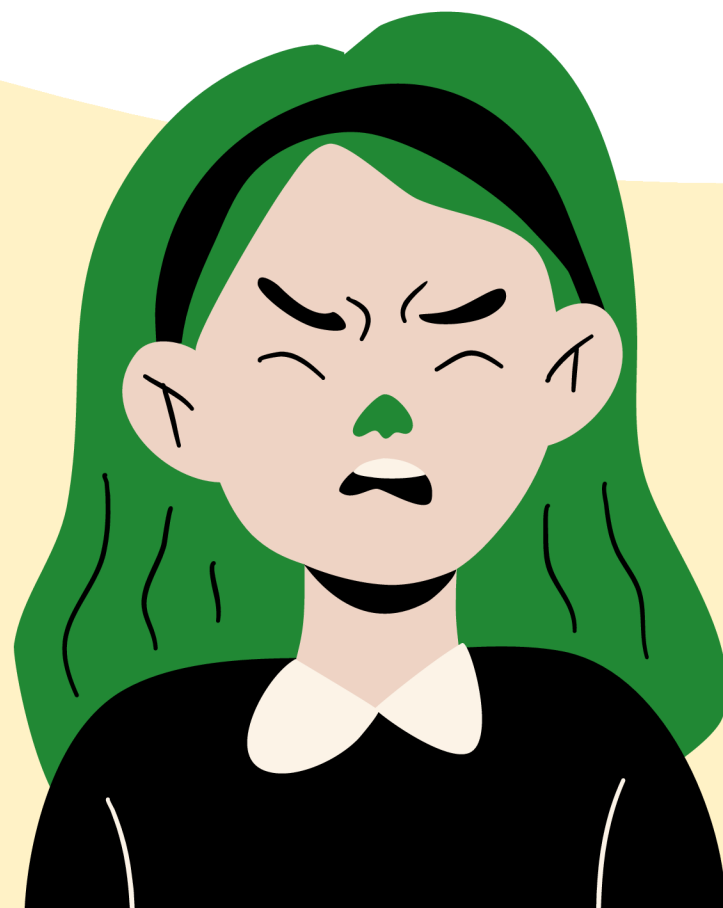


How Are You Feeling Today?

臉部情緒分析

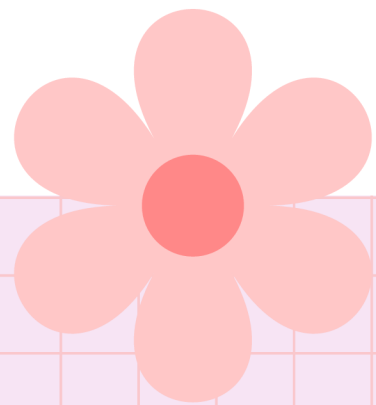
第四組: 王祁騫、黃紹綱、林紘宇、何書維、朱筱琪



目錄



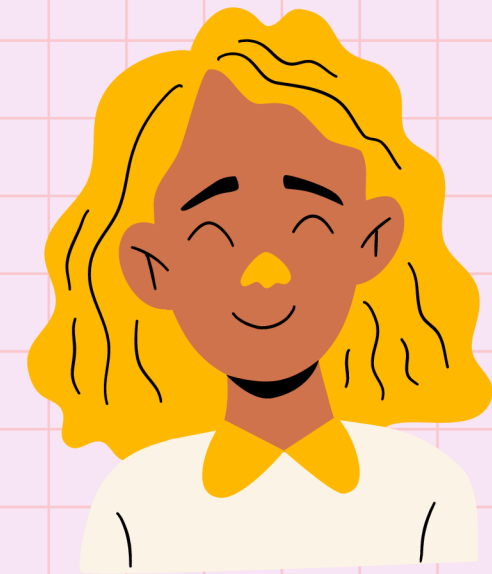
What type
of emotion
can you
identify with?



專案動機



系統架構與
部署流程



主要功能
介紹



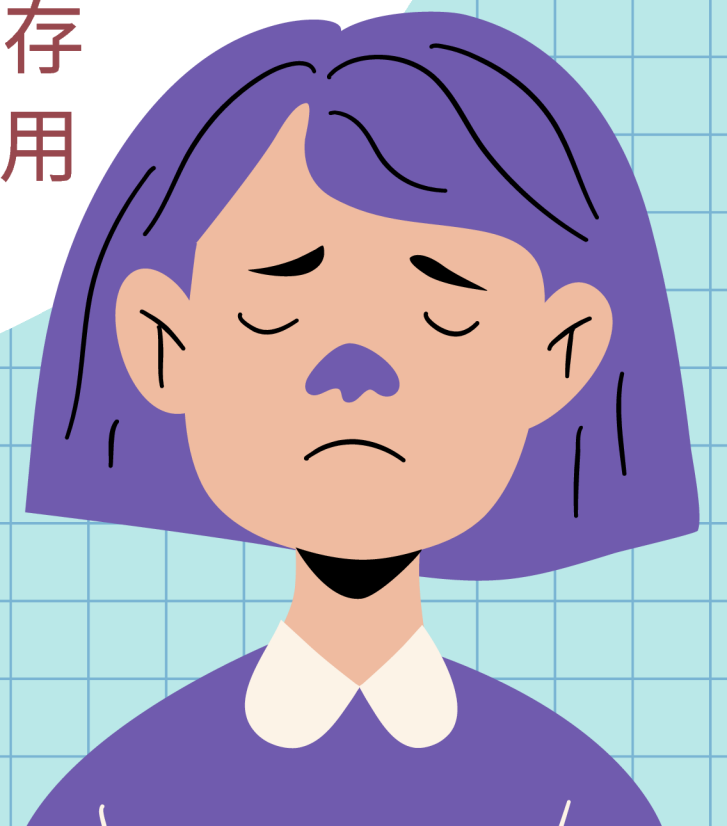
程式碼及
Azurez服務使
用說明



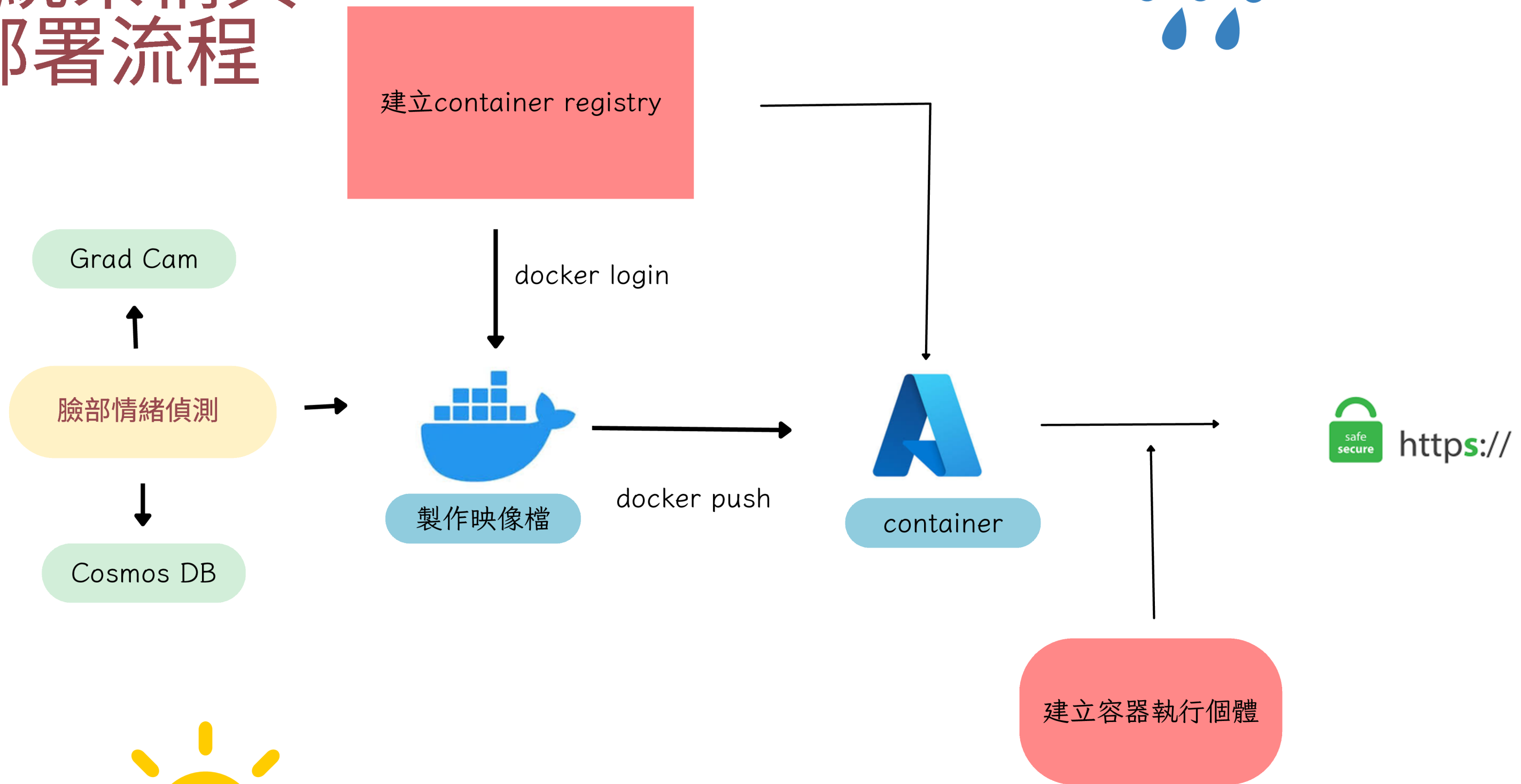
未來展望

專案動機

該專案討論重點是臉部表情辨識在提取GCN特徵後是否能使模型泛化能力更好，由於依靠做圖較難以讓閱讀者理解我們專案所使用的方法，所以建立該網頁的初衷是期望使用者直觀理解本專案納入GCN在臉部特徵的原因、臉部特徵以及各模型辨識效果。將網頁放上雲端則是由於該專案不管是臉部偵測、模型推論、Grad-CAM 生成等等，都較吃效能，所以讓其不受限於硬體環境是其最大需解決問題。Azure Cosmos DB 將當前時間以及當下臉部辨識的結果存入，用意是可在未來對其進行使用者分析，或是方便使用API讀取建立Dashboard等



系統架構與部署流程



主要功能介紹

臉部情緒辨識	Grad-CAM	Azure Cosmo DB
偵測使用者當前的表情，總共有七種情緒可以被偵測 • Surprised • Fearful • Disgusted • Happy • Sad • Angry • Neutral	一種用於可視化卷積神經網路（CNN）決策過程的技術，它可以幫助我們理解模型在進行分類時，到底「看」了輸入圖像的哪些部分	作為紀錄偵測表情的Nosql資料庫服務，Primary key 為偵測系統當下的時間，Value 為使用者被偵測的表情。系統每隔幾秒會自動觸發將表情寫入cosmos db中。



- 在本地端建立檔案的映像檔之後，在Azure架設容器執行個體以及建立能夠自本地端的登入雲端的方法後，將檔案push到雲端。
- 完成映像上傳後，我們建立容器執行個體，將映像部署至雲端執行環境。這樣的設計能夠將所有需大量運算資源的作業，例如情緒推論與模型視覺化，轉移至雲端執行，減輕本地負擔並提升系統效能與擴展性。
- 此外，部署後的容器可透過公開的HTTPS 端點與外部服務進行串接，使整體系統具備即時性、可擴展性與部署彈性，適合長期運作與服務擴充需求。

Azure 工具用法說明

程式碼說明-app.py

```
@app.route("/upload_frame", methods=["POST"])
def upload_frame():
    try:
        # 1. 讀取並解碼
        img_bytes = request.get_data()
        img = cv2.imdecode(np.frombuffer(img_bytes, np.uint8), cv2.IMREAD_COLOR)
        if img is None:
            return "Bad image", 400

        h, w = img.shape[:2]
        mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=img)
        detection_result = landmarker.detect(mp_img)

        if not detection_result.face_landmarks:
            return jsonify({"error": "No face detected"}), 400

        # 2. 取第一張臉，計算 ROI
        lm = detection_result.face_landmarks[0]
        pts = np.array([(p.x*w, p.y*h) for p in lm])
        x0, y0 = pts.min(axis=0).astype(int) - 20
        x1, y1 = pts.max(axis=0).astype(int) + 20
        x0, y0 = np.clip([x0, y0], 0, [w, h])
        x1, y1 = np.clip([x1, y1], 0, [w, h])
        roi = img[y0:y1, x0:x1]

        # 3. 前處理 + 推論
```

```
# 3. 前處理 + 推論
A, X = get_adjacency_matrix(lm)
x, ei, ew, batch, img_tensor = process_for_model(roi, A, X)

with torch.no_grad():
    out = current_model(x, ei, ew, batch, img_tensor)
    logits = out[0] if isinstance(out, (tuple, list)) else out
    probs = torch.softmax(logits, dim=1).squeeze() # Tensor
    pred = int(torch.argmax(probs).item())
    label = label_map[pred]
    probs = probs.cpu().tolist() # → list[float]

# 4. 畫框+文字
vis = img.copy()
cv2.rectangle(vis, (x0, y0), (x1, y1), (0, 255, 0), 2)
cv2.putText(vis, "Generate docs (Ctrl+.)",
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)

# 5. 寫入全域 state
with state_lock:
    state["frame_vis"] = vis
    state["latest_prob"] = probs
    state["predicted_label"] = label
    state["latest_detection"] = {"face_landmarks": lm,
                                "bbox": (x0, y0, x1, y1)}
    state["last face roi"] = roi.copy()
```

程式碼說明-app.py

```
# 4. 畫框+文字
vis = img.copy()
cv2.rectangle(vis, (x0, y0), (x1, y1), (0, 255, 0), 2)
cv2.putText(vis, label, (x0, y0 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)

# 5. 寫入全域 state
with state_lock:
    state["frame_vis"] = vis
    state["latest_prob"] = probs
    state["predicted_label"] = label
    state["latest_landmarks"] = lm,
    state["bbox"] = (x0, y0, x1, y1)
    state["last_face_roi"] = roi.copy()

prob_out = [{"label": label_map[i], "probability": round(p*100, 2)}
            for i, p in enumerate(probs)]
return jsonify({"predicted_label": label,
               "probabilities": prob_out})

except Exception as e:
    import sys, traceback
    traceback.print_exc(file=sys.stderr)
    return jsonify({"error": str(e)}), 500
```

程式碼說明 - capture_client.py

```
capture_client.py
import os, cv2, time, requests

# —— 環境參數 ——
SERVER = os.getenv("SERVER_URL", "http://4.156.184.85:8080/")
TOKEN = os.getenv("INGEST_TOKEN", "changeme")
CAM_ID = int(os.getenv("CAM_ID", 0)) # 通常第一隻鏡頭就是 0
INTERVAL = float(os.getenv("INTERVAL", 0.04)) # ≈25 fps

# —— 攝像頭嘗試用 DirectShow (CAP_DSHOW) ——
# 如果你的 Python 版本有支援，也可改成 CAP_MSMF, CAP_DSHOW, CAP_VFW 等
cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
if not cap.isOpened():
    raise RuntimeError(f"無法開啟攝像頭 (index={CAM_ID})，請確認：\n"
        " 1. 沒有其他程式在用相機\n"
        " 2. CAMERA ID 是否正確 (可試 0,1,2 ...) \n"
        " 3. 驅動程式是否安裝正常")

headers = {
    "Authorization": f"Bearer {TOKEN}",
    "Content-Type": "image/jpeg"
}

print(f"[INFO] 開始擷取攝像頭 (index={CAM_ID})，目標伺服器：{SERVER}/upload_frame")
while True:
    ok, frame = cap.read()
    if not ok:
        print("[WARNING] 無法抓到 frame，重試中...")
        time.sleep(0.1)
        continue

    # 80% JPEG 品質，大約 30-40 kB 左右
    _, buf = cv2.imencode('.jpg', frame, [int(cv2.IMWRITE_JPEG_QUALITY), 80])

    try:
        # 把 JPEG bytes 直接 POST 到 /upload frame
```

```
print(f"[INFO] 開始擷取攝像頭 (index={CAM_ID})，目標伺服器：{SERVER}/upload_frame")
while True:
    ok, frame = cap.read()
    if not ok:
        print("[WARNING] 無法抓到 frame，重試中...")
        time.sleep(0.1)
        continue

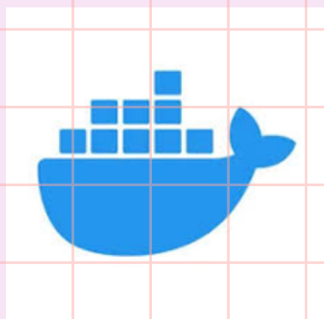
    # 80% JPEG 品質，大約 30-40 kB 左右
    _, buf = cv2.imencode('.jpg', frame, [int(cv2.IMWRITE_JPEG_QUALITY), 80])

    try:
        # 把 JPEG bytes 直接 POST 到 /upload_frame
        resp = requests.post(f"{SERVER}/upload_frame",
            data=buf.tobytes(),
            headers=headers,
            timeout=1)
        if resp.status_code != 200:
            print(f"[ERROR] 上傳失敗 (HTTP {resp.status_code}): {resp.text}")
    except requests.exceptions.RequestException as e:
        # 網路連線有問題，就印訊息，但不讓程式停掉
        print(f"[WARNING] 網路傳送失敗：{e}")

    time.sleep(INTERVAL)
```

程式碼說明Cosmos DB

```
1 def upload_emotion_log_to_cosmos(df, endpoint, key, database_name, container_name):
2
3     client = CosmosClient(endpoint, key)
4
5     database = client.create_database_if_not_exists(id=database_name)
6
7     container = database.create_container_if_not_exists(
8         id=container_name,
9         partition_key=PartitionKey(path="/id"),
10        offer_throughput=400
11    )
12
13    for _, row in df.iterrows():
14        item = {
15            'id': str(row['id']),
16            'face': row['face']
17        }
18        try:
19            # response = container.create_item(body=item)
20            print(f"成功上傳: {item}")
21        except Exception as e:
22            print(f"上傳失敗: {item}, 錯誤: {e}")
```



Docker file

```
1 FROM --platform=linux/amd64 python:3.9
2
3
4 RUN apt-get update && \
5     DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
6         libgl1 libglu1-mesa-dev && \
7         rm -rf /var/lib/apt/lists/*
8
9 WORKDIR /app
10
11 COPY requirements.txt ./
12 COPY Download.py ./
13 COPY load_model.py ./
14 COPY app.py ./
15 COPY models.py ./
16 COPY face_landmarker.task ./
17 COPY .env ./
18 COPY model/model_Net_VGG.pth /app/model/
19 COPY mlflow /app/mlflow/
20
21 COPY static /app/static/
22 COPY templates /app/templates/
23 COPY utils /app/utils/
24
25 RUN pip install --no-cache-dir --upgrade pip && \
26     pip install --no-cache-dir -r requirements.txt
27
28 RUN python load_model.py
29
30 ENV PORT=8080
31 ENV INGEST_TOKEN=changeme
32
33 EXPOSE 8080
34
35 ENTRYPOINT ["python", "app.py"]
36
37
```

未來展望

多指標辨別情緒

- Video Indexer能同時收集臉部表情、語音轉錄與情緒分數
- 利用Azure Video Indexer / Speech SDK 提供的時間軸特徵寫入 Azure Blob 整合模型→ 用 Azure Machine Learning 訓練 Cross-Modal Transformer。

模型持續學習

- 持續學習：利用 Pipeline Component 把「最新一週 inference 資料」併入模型的訓練資料內，使模型架設在雲端期間仍然能繼續加強判斷力

影像連接延遲優化

- 可能嘗試換種方式連接本機端攝像頭，即時傳輸訊息



Thank you for
listening

